Experiment 2

Implementation and Comparison of Linear, Multiple Linear and Logistic Regression models

Importing all the prerequisite libraries

```
import pandas as pd
import matplotlib.pyplot as plt
```

Linear Regression

```
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing(as_frame = True)
df = housing.frame
df.shape

#choosing input and target columns
X = df[['MedInc']]
y = df['MedHouseVal']
```

```
(20640, 9)
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (16512, 1)
Testing data: (4128, 1)
```

```
from sklearn.linear_model import LinearRegression
model = LinearRegression()

model.fit(X_train, y_train) #training the model
print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

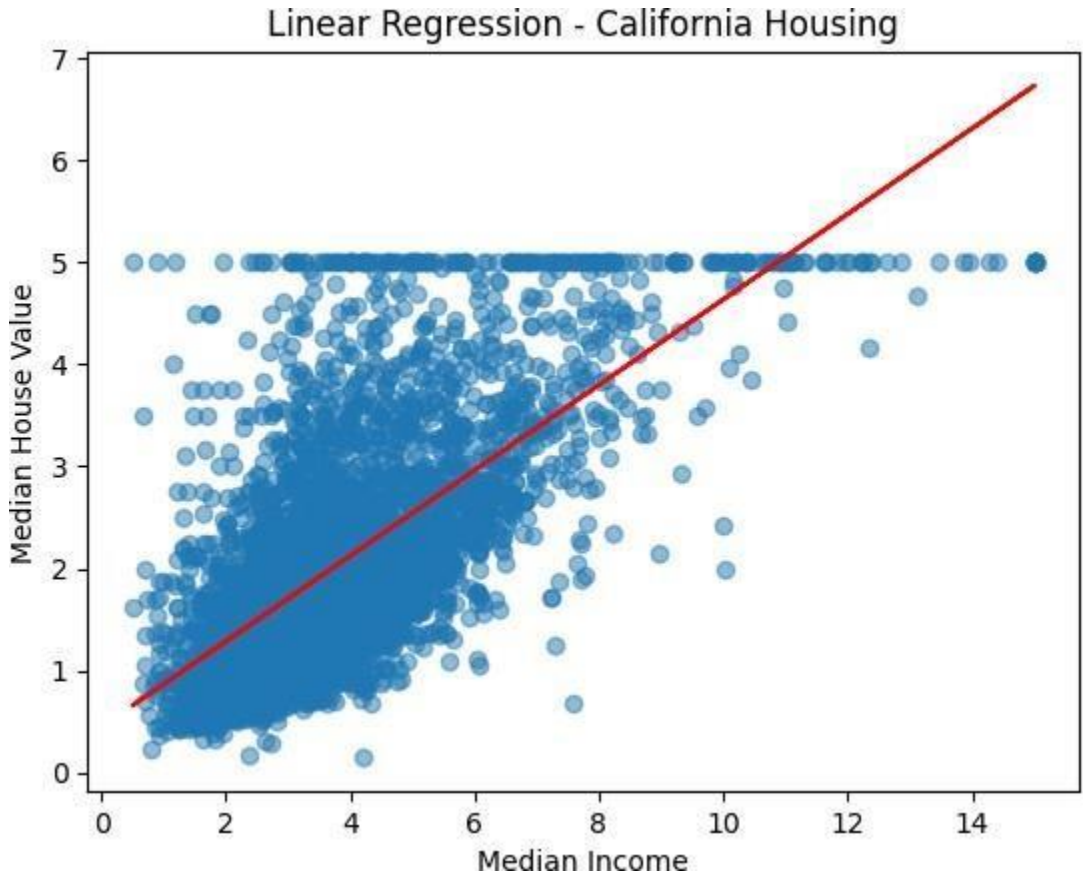
```
Coefficient: [0.41933849]
Intercept: 0.4445972916907879
```

```
y_pred = model.predict(X_test)

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
```

```
mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
MAE: 0.629908653009376
RMSE: 0.8420901241414455
R² Score: 0.45885918903846656
```

```
plt.scatter(X_test, y_test, alpha=0.5)
plt.plot(X_test, y_pred, color='red')

plt.xlabel('Median Income')
plt.ylabel('Median House Value')
plt.title('Linear Regression - California Housing')

plt.show()
```



Multiple Linear Regression

```
df = pd.read_csv("student-mat.csv"  sep=";")
```

```
df.shape

df_encoded = pd.get_dummies(df, drop_first=True)
X = df_encoded.drop('G3', axis=1)
y = df_encoded['G3']
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (316, 41)
Testing data: (79, 41)
```

```
model = LinearRegression()
model.fit(X_train, y_train)

print("Coefficient:", model.coef_)
print("Intercept:", model.intercept_)
```

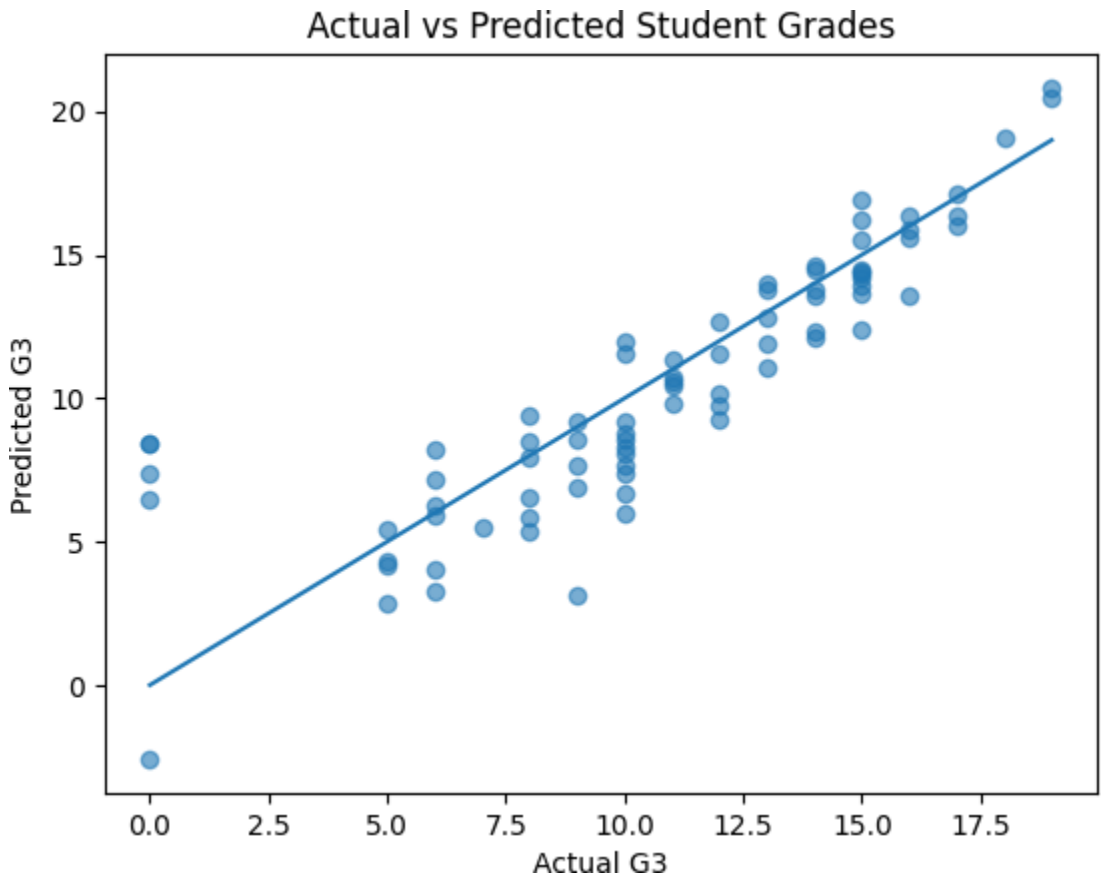
Show hidden output

```
y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)
rmse = mean_squared_error(y_test, y_pred) ** 0.5
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
MAE: 1.6466656197147511
RMSE: 2.3783697847961367
R² Score: 0.7241341236974022
```

```
plt.scatter(y_test, y_pred, alpha=0.6)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()]
)

plt.xlabel("Actual G3")
plt.ylabel("Predicted G3")
```

```
plt.title("Actual vs Predicted Student Grades")

plt.show()
```



Logistic Regression

```
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()

X = cancer.data
y = cancer.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (455, 30)
```

```
Testing data: (114, 30)
```

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=10000)
model.fit(X_train, y_train)

print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
```

```
Coefficients: [[ 1.0274368   0.22145051 -0.36213488  0.0254667  -0.15623532 -
  -0.53255786 -0.28369224 -0.22668189 -0.03649446 -0.09710208  1.3705667
  -0.18140942 -0.08719575 -0.02245523  0.04736092 -0.04294784 -0.03240188
  -0.03473732  0.01160522  0.11165329 -0.50887722 -0.01555395 -0.016857
  -0.30773117 -0.77270908 -1.42859535 -0.51092923 -0.74689363 -0.10094404]]
Intercept: [28.64871395]
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

```
Accuracy: 0.956140350877193
Precision: 0.9459459459459459
Recall: 0.9859154929577465
F1 Score: 0.9655172413793104
```

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(
    y_test,
    y_pred,
    display_labels=cancer.target_names
)

plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```


Confusion Matrix - Logistic Regression
malignant
benign
True label
39
4
1
70
malignant
benign
Predicted label
70
60
50
40
30
20
10